

Dependency and Scope, formalised

A report on the Isabelle/HOL development

11 September 2026

Abstract

This report describes an Isabelle/HOL formalisation of Halvorson’s *Dependency and Scope: On the Translation Between Lemmon and Fitch Proofs*. The development is 32,372 lines across 49 theories, with 1,234 lemmas, theorems and corollaries, and no admitted proofs. It is built in two layers: a compact reconstruction of the calculi, in which the mathematics is developed, and an exact layer reproducing the datatypes and twenty-one rule checks of the *How Logic Works* proof checker, against which the paper’s algorithms were implemented. The principal positive result is a construction theorem at the exact types: every nonempty correct Lemmon proof is translated by the checked translation into a Fitch proof that passes every check, carries the same conclusion, and needs no additional premise. The principal negative results are Theorem 10 and Conjectures 27 and 28, each refuted at both layers. Along the way the formalisation corrected four claims in the paper, identified fourteen defects in the supplied Haskell, two of which affect soundness, and isolated two design decisions in the original checker on which the main theorem turns out to depend.

Contents

1	Introduction	2
2	The two calculi	3
2.1	Lemmon proofs	3
2.2	Fitch proofs	3
2.3	The erasure	4
3	What “correct” means	4
4	Semantics and soundness	4
5	The erasure and the dependency column	5
6	The line-by-line translation	5
7	The unfolding construction, compact	6
8	The construction at the exact types	6
8.1	Unfolding	6
8.2	Renaming	7
8.3	Layout	7

8.4	Structural well-formedness	7
8.5	Rule transfer through the erasure	7
8.6	The scope check	8
8.7	The construction theorem	8
8.8	One hypothesis	9
9	The impossibility results	9
9.1	Three obstructions	9
9.2	At the compact types	9
9.3	At the exact types	9
10	What the formalisation found	10
10.1	Corrections to the paper	10
10.2	Defects in the supplied Haskell	11
10.3	Two things the main theorem depends on	11
11	Validation	12
12	What is not established	12
A	Theory map	13
B	Claim to theorem	14

1 Introduction

Lemmon and Fitch notations present natural deduction differently. A Lemmon proof is a numbered list; each line carries a formula, a justification citing earlier lines, and an explicit set of the assumptions it depends on. A Fitch proof is a nested structure; assumptions open boxes, and the discharge of an assumption closes one. The paper asks how the two are related: whether one can always be translated into the other, whether the translation can be made line-by-line, and what the dependency column is for.

Formalising a paper of this kind involves a choice that is worth stating plainly at the outset. One can formalise the *mathematics* — reconstruct the two calculi in whatever form is most convenient and prove the theorems about them — or one can formalise the *implementation* — reproduce the concrete rule set of the checker the paper’s algorithms were written against, and prove the theorems about that. These are different obligations, and a theorem about the first is not a theorem about the second. The rule rosters differ, the correctness predicates differ, and, as §9.3 records, the two do not embed in one another.

This development does both, in two layers:

The compact layer reconstructs the calculi as `fm`, `just`, `pline`, `fitch_item`. It is where the mathematics is developed: the erasure, the unfolding construction, the impossibility results.

The exact layer reproduces the datatypes and the twenty-one rule checks of the *How Logic Works* checker under `HL_`/`h1` names, including its representation choices down to the empty-string marker for a vacuous quantifier. This is the authoritative specification for the executable system, and the layer in which the final results are stated.

The report is organised by subject rather than by layer. §2 fixes the objects and §3 the several notions of correctness, which are not interchangeable. §4 covers soundness, §5 the erasure and the dependency column, §6 the line-by-line translation, §7 the unfolding construction in the compact layer and §8 at the exact types, which is the bulk of the recent work. §9 gives the negative results. §10 collects what the formalisation found: corrections to the paper, defects in the supplied code, and two observations about the checker’s design. §11 describes how the development is checked and §12 what it does not establish.

2 The two calculi

2.1 Lemmon proofs

A Lemmon line is a quadruple: a dependency set, a line number, a formula, and a justification. Writing the dependency set on the left, as the paper does, a proof of $Q \rightarrow (P \rightarrow (P \wedge Q))$ looks like this.

deps	line	formula	justification
1	(1)	P	A
2	(2)	Q	A
1,2	(3)	$P \wedge Q$	$\wedge I$ 1,2
2	(4)	$P \rightarrow (P \wedge Q)$	CP 1,3
	(5)	$Q \rightarrow (P \rightarrow (P \wedge Q))$	CP 2,4

Line 4 discharges assumption 1 — so 1 leaves the dependency set — and line 5 discharges assumption 2. The justification of a discharging rule names two line numbers: the assumption being discharged, and the line at which the discharge occurs.

In Isabelle the compact form is `pline` with `just` in `LF_Lemmon.thy`; the exact form is `hl_line` with `hl_justification` in `LF_HLW.thy`. The exact justification type has twenty-one constructors, matching the checker: modus ponens and modus tollens, double negation, conditional proof, reductio, the conjunction, disjunction and biconditional rules, the four quantifier rules, the two equality rules, excluded middle, quantifier negation, and an arbitrary propositional-consequence rule `HL_PropTaut`.

2.2 Fitch proofs

A Fitch proof is a list of items, each either a line or a subproof; a subproof carries an assumption and a body which is again such a list. Compact `fitch_item` and `subproof` live in `LF_Fitch.thy`, exact `hl_fitch_item` and `hl_subproof` in `LF_HLW_Fitch.thy`.

Two structural facts about the exact form matter later. Flattening a Fitch proof, `hlFlattenFitch`, produces a list of triples — number, formula, rule — with the boxes’ contents spliced in place, and a subproof contributes its assumption line first. And the well-formedness checker records, for each line, which completed boxes are citable there: a discharge may name a pair (a, c) only when a box with head a and last line c has *closed at that line’s own level*. That is `hlFitchNestingFrom`, and §9 shows it is the engine of all three impossibility results.

2.3 The erasure

The translation from Fitch to Lemmon is the easy direction, and the paper’s Definition 3 gives it: flatten the proof and compute each line’s dependency set from the rule and the sets of the lines it cites. This is `delta` in the compact layer and δ_H (`h1FitchToLemmon`) in the exact one. It is a total function on all Fitch objects, well-formed or not.

3 What “correct” means

The development distinguishes three predicates on exact Lemmon proofs, and the distinction is load-bearing rather than bureaucratic.

`h1Correct` is the faithful reproduction of the supplied checker: every line passes `h1StructureOK` and `h1RuleOK`. It is deliberately *not* repaired, because its purpose is to be what the Haskell does. It permits physically unsorted line lists and does not constrain the dependency set of an excluded-middle line.

`h1VerifiedCorrect` adds dependency closure and a canonical ordering: positive, strictly increasing line numbers.

`h1PaperCorrect` adds the paper’s exact dependency arithmetic, in particular that an excluded-middle line depends on nothing (`h1PaperCorrect_LEM_empty`).

These are strictly ordered, with `h1PaperCorrect_verified` the easy inclusion. The reason to keep three is that the paper’s claims about the dependency column are true of the third and false of the second. The sharpest statement of that is in `LF_HLW_Canonical.thy`:

Theorem (`h1VerifiedCorrect_has_no_dependency_reconstructor`). *There is no function that recovers the dependency sets of every `h1VerifiedCorrect` proof from its stripped lines.*

It is proved by exhibiting two accepted proofs with identical stripped data and different dependency sets, so no function could send the one to both (`h1VerifiedCorrect_does_not_determine_dependencies`). This is what forces the third predicate to exist.

On the Fitch side there is a similar layering: `h1FitchWellFormed` is the faithful reproduction of the checker’s structural test, `h1FitchVerified` adds genuine nesting, premise closure, a top-level conclusion and correctness of the Lemmon erasure, and `h1FitchCorrect` adds the scope-based rule check. §10 explains why the faithful version had to be left alone and the repairs put in the layers above it.

4 Semantics and soundness

Both layers have a semantics and a soundness theorem, so that “correct” is answerable to something other than itself.

For the compact calculus, `LF_Semantics.thy` proves `lemmon_semantic_soundness` and `fitch_semantic_soundness`: an accepted proof’s conclusion is a semantic consequence of the assumptions it displays. `derivOK_sound` does the same for derivation trees.

For the exact calculus, `LF_HLW_Semantics.thy` — 2,814 lines, the largest theory in the development — proves `h1RuleOK_sound`: whenever a line passes the checker’s rule test, its formula follows

from the formulas of the lines it cites. This is done rule by rule, and the quantifier rules are where the work is. `hlForallIntroduction_sound` and `hlExistentialElimination_sound` require the eigenconstant conditions to be read correctly, and both rest on soundness results for the checker’s witness inference (`hlWitnessLists_forall_sound`, `hlWitnessLists_exists_sound`) which say that an inferred witness list really does describe an instantiation. `hlPropositionalConsequence_sound` covers the tautology rule by way of valuations, and `hlQuantifierNegationEquivalent_sound` the quantifier-negation rule. `hlVerifiedCorrect_line_sound` assembles these into the statement for whole proofs.

`LF_HLW_Fitch_Semantics.thy` proves `hlFitchVerified_sound` and `hlFitchCorrect_sound` from the displayed outer premises. These use the premise-closure check explicitly; they do not show it redundant.

5 The erasure and the dependency column

Theorem 4. The paper states that δ is total and that a Fitch proof is correct if and only if its erasure is. Totality holds by construction. The forward implication is `theorem_4_forward` in `LF_Delta.thy`, proved from a comparison of Fitch scope with reconstructed dependencies. The converse is *false*, and the witness is the paper’s own Theorem 22 example: erasure discards excess scope, and with it the eigenconstant violation that made the Fitch proof incorrect. This is `theorem_4_backward_fails` in the compact layer and `hl_theorem_4_backward_fails` in the exact one.

Propositions 5 and 7, Corollary 6. `proposition_5` establishes that reconstructed dependencies lie within scope, and may lie properly within it; `corollary_6` exhibits two distinct correct Fitch proofs with the same erasure, so δ is not injective. `proposition_7` is the statement that the dependency column is redundant: from a correct proof with its dependency sets deleted, they can be recomputed. It holds for compact `lemmonCorrect` and, as `hl_proposition_7`, for exact `hlPaperCorrect`, with `hlPaperCorrect_dependencies_unique` giving uniqueness. As noted in §3, it fails for the weaker predicate, and that failure is proved rather than assumed.

6 The line-by-line translation

The paper’s Definition 9 asks when a Fitch proof is a *positional image* of a Lemmon proof: same lines, in the same order, with corresponding rules. `positionalImage` in `LF_Conjecture27.thy` formalises it, with `lineMatches` pinning each Fitch line’s rule to the corresponding source justification. Assumptions may correspond either to Fitch premises or to box assumptions — without that latitude the notion would be vacuously unsatisfiable.

The direct algorithm `lemmonToFitchDirect` walks the source once and opens a box at each assumption; it is analysed in `LF_Positional.thy` (2,014 lines, 102 results). The main results establish what its output looks like when it succeeds: sorted numbering, premises first, the right citations, a top-level final line, and, by `lemmonToFitchDirect_fitchWF_iff_citations`, that its well-formedness reduces exactly to a citation condition. `lemmonToFitchDirect_positional` in `LF_Direct.thy` shows that when it returns a proof, that proof is a positional image of the source.

The algorithm is guarded by seven checks. One of them, `boxHeadError`, exists because of a defect: without it the algorithm can emit two boxes sharing a last line, which is exactly the

configuration §9 shows no well-formed Fitch proof can contain. `checks_reject_only_imageless_sources` records the partial converse that was established — all six witnesses the checks reject are genuinely imageless — while being explicit that six witnesses are not all sources. Whether the seven checks are *necessary* as well as sufficient is recorded as an open question rather than claimed.

7 The unfolding construction, compact

When no positional image exists, the paper’s Definition 21 gives a different route: unfold the Lemmon proof into a derivation tree rooted at its conclusion, then emit a Fitch proof from the tree. Four obligations follow, which the paper labels L1–L4.

- L1** Unfolding terminates on every correct nonempty source. `unfolding_terminates` and L1 in `LF_Unfold.thy`. The recursion is well-founded because citations point to smaller numbers, which is `proposition_18`.
- L2, L3** Citations precede their uses and are in scope; premises are emitted once, at the top. `L2_L3` in `LF_WellFormed.thy` (1,906 lines).
- L4** Each rule transfers: the emitted line satisfies the corresponding Fitch rule, including the sub-proof endpoints the discharging rules need and the eigenconstant conditions. `L4_derivation` in `LF_Check.thy` (2,465 lines).

Two supporting results are the paper’s Lemma 23 and Corollary 24: renaming a constant to a globally fresh one preserves a derivation (`renaming`), and leaves the open assumptions unchanged when the old constant does not occur in them (`renaming_open`). These are what make the eigenconstant repair legitimate.

The construction theorem for the compact calculus is `conjecture_26` in `LF_Conjecture.thy`, with `translation_total` and the sequent corollary `conjecture_26_sequent`. It is a *corrected* replacement for the paper’s Conjecture 26 in one respect: the original repairs only universal introduction, and the repair must also be applied to existential elimination, whose witness carries the same kind of freshness condition. This is discussed in §10.

8 The construction at the exact types

Everything in §7 is about the compact calculus. The corresponding statements for the exact checker do not follow from it — the datatypes, the rule roster and the emitter are all different — and establishing them is the bulk of the recent work. It proceeds in five steps.

8.1 Unfolding

`h1UnfoldDerivation_succeeds` supplies sufficient fuel by citation rank and `h1ToDerivation_total` gives successful unfolding for every nonempty verified source; `h1ToDerivation_correct` then proves the resulting tree correct, with the same conclusion and no additional premises, for all twenty-one rules (`h1VerifiedCorrect_toDerivation`, `h1PaperCorrect_toDerivation`).

8.2 Renaming

`hlDerivationOK_rename` proves that renaming to a fresh constant preserves every exact derivation rule, quantifier witness selection, equality and propositional consequence included; `hlRenameDerivation_open_unchanged` is the Corollary 24 analogue. The hypotheses exclude the checker’s empty-name sentinel, which is not decoration: renaming that marker would corrupt the witness-inference logic.

8.3 Layout

`LF_HLW_Layout.thy` proves the emitter allocates a consecutive interval of line numbers (`hlAllocated`), `LF_HLW_Fuel.thy` that the fuel supplied is always sufficient, and `LF_HLW_Conclusion.thy` that the emitted proof concludes with the tree’s formula.

8.4 Structural well-formedness

`LF_HLW_Nesting.thy` proves `hlEmitDerivationFuel_nesting` by well-founded induction on derivation size across all twenty-one rules: the emitted fragment satisfies the genuine nesting predicate, so every discharge names a box that really closed at the right level. Ordinary citation visibility then needs no second induction: `hlFitchNestingFrom_scope` derives it, because the scope check is the weaker of the two — after a box it makes the box’s assumption and last line visible, which nesting does not, and it inspects no discharge pairs at all.

8.5 Rule transfer through the erasure

This is the long pole. What must be shown is that erasing the emitted proof gives a *correct Lemmon proof*, which means each of the three checks in `hlVerifiedCorrect`:

- `hlFitchToLemmon_canonicalOrder` — the erased numbers are the flattened Fitch numbers, allocated strictly increasingly;
- `hlFitchToLemmon_dependencyClosed` — which holds of *every* Fitch proof, with no hypothesis, because `hlFitchDependenciesOf` computes exactly the union each rule demands;
- `hlFitchToLemmon_structureOK` — from the fact that citations point strictly backwards, which needs only sortedness;
- and rule correctness, `hlEmitDerivationFuel_ruleOK`, over all twenty-one rules.

Three things carry the last of these. First, `hlFitchToLemmon_references`: the partial environment the erasure held when it computed a line gives, at every line that rule may cite, the same answer as reading the whole finished proof back — later lines cannot interfere because their numbers are larger and citations point backwards. Second, `hlEmitDerivationFuel_deps`: an emitted line’s dependencies are exactly the environment image of its subderivation’s open assumptions. The load-bearing step there is `hlEnvImage_drop`, which says that because a box head is freshly allocated, subtracting it removes exactly the discharged assumption and never an older one — the same hazard class as the `boxHeadError` defect of §6.

Third, the eigenconstant obstruction, which was real. At the level of the derivation, `hlForallIntroStep` and `hlExistsElimStep` state freshness against the *derivation’s* open assumptions; `hlRuleOK` states it against the *proof’s* assumption lines. That gap is

closed, given the dependency equation, by `hlAssumptionConstants_envImage` together with `hlReferencedConstants_envImage`.

8.6 The scope check

`hlFitchCorrect` requires more than the above: the eigenconstant conditions must hold against the assumptions genuinely in Fitch *scope*, which is `hlCorrectG (hlScopeSrc F)`. This does not follow from what precedes it. A line's scope contains every root premise and every enclosing box head, so it is in general *larger* than that line's dependency set, and the antitonicity lemmas in `LF_HLW_Rule_Transfer.thy` run the wrong way.

`hlEmitDerivationFuel_scopeOK` proves it by a second induction over the emitter carrying three further invariants: that the assumption constants of the scope set are contained in the constants of the emitter's own `scope` list; that every number in the scope set is below the next free line, so a freshly allocated box head is never already in scope; and `hlScopeBelow`, that every constant the scope holds is shorter than the name the next repair will mint. Nineteen of the twenty-one rules reduce immediately, since only `HL_ForallIntro` and `HL_ExistsElim` consult the scope at all. What makes the remaining two go through is discussed in §10.

The freshness bookkeeping is in `LF_HLW_Fresh.thy`: `hlNamesBelow base cnt d` says every constant in *d* is shorter than the name the next repair will mint, which survives each repair because a repair adds one name of exactly that length and increments the counter. That is what the counter in `hlFreshIndex` is for, and the invariant transfers to every subderivation by `hlEmitDerivationFuel_count_le` and `hlDerivationConstants_sub`.

8.7 The construction theorem

Theorem (`hlPaperCorrect_toFitch`, `LF_HLW_Construct.thy`). *For every nonempty proof P satisfying `hlPaperCorrect` there are a route and an F with*

$$\begin{aligned} \text{hlLemmonToFitchChecked } P &= \text{Inr}(\text{route}, F), & \text{hlFitchCorrect } F, \\ \text{hlFitchConclusion } F &= \text{hlConclusion } P, & \text{hlConclusion}(\delta_H F) = \text{hlConclusion } P, \\ \text{set}(\text{hlFitchPremises } F) &\subseteq \text{set}(\text{hlOpenPremises } P). \end{aligned}$$

The route is existential because the checked operation tries the direct algorithm first and falls back to the tree construction; since Conjecture 27 is false, no single direct algorithm can be claimed, and the theorem is about the translation as a whole. This is strictly stronger than the earlier `hlViaTree_accepted` and `hlLemmonToFitchChecked_accepted`, which guarantee any *particular* accepted output but cannot rule out the translator rejecting a good source.

Assembling it required discharging the inductions' hypotheses at the root: `hlEmitCtx` bundles the requirements that the layout environment cover every open assumption, that each environment line be its own sole dependency, and that each carry that assumption's formula. All three come from the premise layout. Premise closure (`hlFitchPremiseClosed`) follows because the conclusion line's dependencies are the environment image of the tree's open assumptions, every one of which is a premise line; the case of a derivation that is a single premise leaf is separate, since then the last line is itself a premise line.

8.8 One hypothesis

The construction theorem needs one side condition on the derivation tree: no assumption label may carry two different formulas (`hlAssumptionsFunctional`). `hlEnvironmentLine` returns the *first* environment entry with a given label, so without this the environment need not describe the tree’s open assumptions at all. It is automatic for anything arising from a proof — a label is a line number and a line carries one formula — and `hlToDerivation_functional` discharges it, since the unfolding places the open assumptions inside `hlSourceAssumptions`, which is functional because `hlLookupLine` is. It therefore does not appear in the theorem above, but it is a genuine condition on arbitrary `hl_derivation` values.

9 The impossibility results

9.1 Three obstructions

The paper’s Theorem 10 says some correct Lemmon proofs have no positional Fitch image, and Conjectures 27 and 28 propose weaker forms that might survive. All three are refuted, and there are exactly three obstructions, which the development keeps distinct because they are genuinely different phenomena.

Obstruction	Witness	Refutes
Discharges do not nest	<code>ex11</code>	Theorem 10
A line is trapped in a box	<code>ex12</code>	Theorem 10
Two discharges share a last line	<code>c27</code>	Conjectures 27, 28

The third is worth emphasising, because it is easy to mistake for one of the others. In `c27` the two boxes nest perfectly and no line is trapped; what fails is that both conditional proofs discharge *from the same line*, one releasing assumption 1 and the other assumption 2, so a Fitch proof would need two boxes ending at the same place.

9.2 At the compact types

`theorem_10_ex11`, `theorem_10_ex12` and `theorem_10` in `LF_Theorem10.thy` are genuine quantified nonexistence statements, not observations that one algorithm returns an error. They rest on the positional apparatus of `LF_Span.thy` and `LF_Positional.thy`: the compact flatten records, for each line, the *path* of boxes containing it, and the arguments are prefix reasoning over those paths, with `subs_span` as the central lemma. `conjecture_27_false` and `conjecture_28_false` in `LF_Conjecture27.thy` do the same for the conjectures, the first quantifying over all injective renumberings, so it covers the permitted permutations rather than one of them.

9.3 At the exact types

Carrying these to the exact types looked like the hardest part of the project, and the two obvious routes both have problems.

A *bridge* between the layers would transport the results, but it must run from exact to compact — to show no exact image exists, assume one and map it down — and in that direction it cannot be total: `HL_MT`, `HL_LEM`, `HL_PropTaut` and `HL_QN` have no compact counterpart, and

the last is an arbitrary propositional-consequence rule. (The other direction is nearly total: `BotI` maps to `HL_PropTaut [i,j]`, `Reit` to `HL_PropTaut [i]`, and the split eliminations collapse onto the single exact ones.) A witness-local bridge would have sufficed, since `lineMatches` pins an image’s rules to the source’s, but that still leaves relating two independently written well-formedness predicates. *Porting* the positional apparatus was the alternative, at the cost of most of a 2,000-line theory.

Neither was needed. The exact layer already carries the same content in a different form, because `hlFitchNestingFrom` admits a discharge citation only when the pair is a box that has closed at the citing line’s own level, and tracks a visible set that a closed box’s interior never re-enters. Three structural facts follow directly, by the induction that `hlBoxSpans` itself follows:

`hlBoxSpans_nested` Box spans never cross: if $a < a' \leq c$ for spans (a, c) and (a', c') , then $c' \leq c$.

`hlFitchNestingFrom_no_cite_into_box` A line numbered past a box’s last line cannot cite a number inside it. The induction carries the invariant that the visible set holds only numbers earlier than the fragment.

`hlBoxSpans_last_lt`, `hlBoxSpans_same_last` A box ends strictly before the proof containing it, so two distinct boxes never share a last line.

Each obstruction is then one of these. `hl_theorem_10_ex11` closes at once: the two conditional proofs would need boxes spanning 1 to 3 and 2 to 4, and $1 < 2 \leq 3$ forces $4 \leq 3$. `hl_theorem_10_ex12` is a little longer. `hlSharedDischarge_no_image` is again immediate, and `hl_conjecture_27_false` follows from it by `hlSharedDischarge_permute`: sharing a discharge is a property of the proof rather than of the page, so it survives every injective renumbering.

10 What the formalisation found

10.1 Corrections to the paper

1. **Theorem 4’s converse is false.** Retain totality and the forward implication; the converse is refuted by the paper’s own Theorem 22 witness.
2. **Proposition 5 needs a corrected notion of scope.** Literal Definition 2 omits outer premises from the assumptions in scope, which makes Proposition 5 false already at a premise line. `scopeOf` includes them.
3. **Definition 17’s graph orientation must be stated explicitly.** The paper writes cited $\rightarrow$ citing edges, so unfolding from the conclusion traverses the reverse relation. The code’s backward recursion is the intended algorithm, not a literal traversal of the written edges.
4. **Conjecture 26 needs both eigenconstant repairs.** The original repairs universal introduction only. Existential elimination’s witness carries the same kind of freshness condition and needs the same treatment; the proved compact result is a corrected construction theorem rather than the original statement.

A fifth item is a matter of interpretation rather than error: the paper’s later remarks about normalisation do not amount to a retract on raw proof objects, since unfolding can duplicate lines, insert reiterations, rename constants and omit lines irrelevant to the conclusion. The revised manuscript withdraws that claim.

10.2 Defects in the supplied Haskell

Fourteen findings in the supplied source are recorded in `HASKELL_AUDIT.md`, together with two in the code Isabelle generates. Most of the fourteen are partiality: omitted `Boolean` cases in six `ProofTypes` functions, `Iff` missing from `ModelSemantics.eval` and from `PropDNF`, six justification constructors unhandled in `JsonToPipe`. Two are more serious.

A structural check that does not check order. `LemmonChecker.checkStructure` compares cited line *numbers* with the current number rather than checking physical proof order. A first line numbered 10 can cite a later line numbered 5 and evade the intended acyclicity check. This is why `hlVerifiedCorrect` exists as a separate predicate above the faithful `hlCorrect`.

A soundness bug. `FitchTypes.fitchWellFormed` checks scope and citations but not rule correctness, eigenvariables, the existence of the subproofs a discharge refers to, or a root-level final conclusion. The last of these is the soundness bug. A proof consisting of one bare subproof therefore has no premises and reports the subproof’s *assumption* as its conclusion — so it “proves” an arbitrary formula from nothing. The witness is `hlOpenAssumptionFitch`, and `hlOpenAssumptionFitch_fails_only_root_scope` shows by evaluation that it passes every other condition, including the scope-based rule check. The faithful `hlFitchWellFormed` reproduces the defect deliberately; the condition was added to `hlConcludesAtTop`, which the layers above require. The same repair in the compact layer turned a refuted soundness claim into the proved `fitch_semantic_soundness`.

A translation bug. `FitchConvert.toDerivation` computes one global set of discharged assumptions. If an assumption is discharged on a line that is dead with respect to the selected conclusion, the tree still labels that assumption as discharged rather than as a premise; the shared-discharge proof then loses an open premise and the emitter cites line 0. Isabelle corrects this with `hlClassifyAssumptions`, whose bound set follows actual derivation ancestors.

10.3 Two things the main theorem depends on

Two design decisions in the original system turned out to be load-bearing in ways that are not obvious from reading it.

The witness inference draws candidates from the premise. For the scope check of §8.6 to go through, an inferred eigenconstant must lie in the set the repair empties of scope constants — namely the constants of the conclusion that are missing from the goal. It does, and the reason is in the checker: `hlWitnessLists` takes its candidate constants from `hlConstantsInFormula q`, the constants of the *premise*. So an inferred witness always occurs in the premise and, having been abstracted away, never in the conclusion. Had the candidates been drawn from anywhere else, the scope check would not follow, and this would be a gap rather than a theorem. The formal statements are `hlForallIntroStep_witness` and `hlExistsElimStep_witness`.

`hlConcludesAtTop` is not presentational. The requirement that a subproof’s body end with a line rather than another subproof looks like a matter of style. Without it, a box whose body ends with a box *inherits that box’s last line*, and `hlBoxSpans_same_last` is false — so Conjectures 27 and 28 would not be refutable by this route. The condition the nesting checker imposes on every

body is exactly what excludes that case. It is also, as above, what repairs the soundness bug in `fitchWellFormed`. One condition is doing two quite different jobs.

11 Validation

Proof checking. The complete development processes from Isabelle’s plain HOL image in about six minutes: `isabelle build -d . -e Lemmon_Fitch`. A source scan finds no `sorry`, no `oops`, no axiomatizations, no oracles and no `quick_and_dirty` settings, and every maintained theory is explicitly imported by the `cap`, so processing the `cap` checks all of it. Of the 1,234 results, 124 are settled by evaluation on concrete witnesses — the counterexamples and the regression suites — and the rest by proof. Isabelle 2025-2; only HOL and HOL-Library are required, and no Archive of Formal Proofs entry.

Differential testing. The development generates a Haskell kernel and an SML kernel by code extraction. The generated checker is compared against the original handwritten one, per line and through its public API rather than through internals: 9,874 comparisons — the supplied 14-proof corpus, twelve hand-written proofs covering the eight rules that corpus never exercises, and 1,728 systematic mutants — with 0 verdict disagreements, of which 2,706 exercise the rejection path; 860 of 860 rejection messages reproducing the original text byte for byte; all 80 cases of the supplied regression suite passing with the generated kernel substituted; 26 of 26 public API assertions. The messages come from `hlLineError`, which Isabelle proves reports no error exactly when `hlLineOK` holds, so the diagnostics cannot alter which proofs are accepted. This is testing, not proof — the two can never be proved equivalent, since the Haskell has no formal semantics here — but it is the check that distinguishes a faithful reconstruction from a careful lookalike.

Regression. `LF_HLW_Regression.thy` and `LF_HLW_Delta.thy` evaluate the checker on the configurations that previously escaped it: closed-box escapes, nonexistent discharge pairs, cross-level subproof citations, malformed placements, and the tree route on examples of all twenty-one rules.

12 What is not established

- **Whether `hlCorrect` alone is semantically sound.** Soundness is proved of the stronger `hlVerifiedCorrect`. Settling this needs a witness rather than an argument from the definitions; the `fitchWellFormed` bug above is the standing reason for caution.
- **Whether the seven direct-translation checks are necessary.** Sufficiency is proved. All six witnesses considered are genuinely imageless, but six witnesses are not all sources.
- **Conjecture 28 under unrestricted citation rewriting.** The refutation fixes an explicit formula-, rule- and citation-preserving notion of image, and allows auxiliary lines (`conjecture_28_with_auxiliaries_false`). A broader notion in which source citations may be redirected to auxiliary derivations is untouched, and the original gives no formal definition to appeal to.
- **Graph-theoretic presentation.** Definitions 14–15 and Propositions 18 and 20 are formalised in their operational content — acyclicity as a descending citation measure, scope as nested datatypes — not as a separate general graph theory with an isomorphism theorem.

- **The application layer.** The web application has not been adapted to the generated Aeson and parser interface. This is interface engineering rather than formalisation.

A Theory map

Theory	Lines	Results	Subject
LF_Formula	319	23	compact formulas, renaming, abstraction
LF_Lemmon	449	12	compact Lemmon proofs and checker
LF_Fitch	346	8	compact Fitch syntax, scope paths
LF_Delta	463	38	erasure; Theorem 4, Propositions 5 and 7
LF_Direct	314	5	the direct algorithm and its seven checks
LF_Derivation	583	37	derivation trees; Lemma 23, Corollary 24
LF_Unfold	561	19	unfolding; L1, Proposition 18
LF_Render	236	1	printing
LF_Examples	403	24	the paper's examples; Theorem 22
LF_WellFormed	1906	91	L2, L3
LF_Faithful	712	20	source-to-tree faithfulness
LF_Check	2465	80	L4, rule transfer
LF_Conjecture	228	9	compact construction theorem
LF_Span	577	28	box spans
LF_Positional	2014	102	positional analysis of the direct route
LF_Semantics	854	44	compact semantics and soundness
LF_Theorem10	246	10	Theorem 10, compact
LF_Conjecture27	719	46	Conjectures 27 and 28, compact
LF_HLW_Formula	339	2	exact formulas
LF_HLW	545	3	exact proofs; the twenty-one rule checks
LF_HLW_Canonical	158	12	hlPaperCorrect; Proposition 7
LF_HLW_Fitch	334	5	exact Fitch syntax; nesting
LF_HLW_Scope	291	7	scope records; hlFitchCorrect
LF_HLW_Semantics	2814	92	exact semantics; hlRuleOK_sound
LF_HLW_Fitch_Semantics	41	2	Fitch soundness
LF_HLW_DNF	137	3	propositional normal forms
LF_HLW_Translate	299	4	exact direct translation
LF_HLW_Unfold	663	3	exact unfolding and emitter
LF_HLW_Translation_Proofs	354	18	L1; conditional guarantees
LF_HLW_Derivation	251	10	exact derivation trees
LF_HLW_Faithful	559	15	G1: source to correct tree
LF_HLW_Renaming	390	47	renaming foundations
LF_HLW_Witnesses	199	13	witness inference
LF_HLW_Quantifier_Renaming	149	12	renaming and quantifiers
LF_HLW_Derivation_Renaming	172	18	G2: renaming preserves rules
LF_HLW_Rule_Transfer	320	12	local transfer; antitonicity
LF_HLW_Layout	222	27	G3: allocation
LF_HLW_Fuel	66	3	fuel adequacy

Theory	Lines	Results	Subject
LF_HLW_Conclusion	75	8	conclusion preservation
LF_HLW_Premise_Layout	161	20	premise placement
LF_HLW_Nesting	944	41	T1: structural well-formedness
LF_HLW_Fresh	501	30	the repair and its name bound
LF_HLW_Erasure	3793	95	T2: rule transfer through δ_H
LF_HLW_Construct	3408	55	T3: the construction theorem
LF_HLW_Theorem10	996	35	impossibility at exact types
LF_HLW_Examples	317	21	the paper's examples, exact
LF_HLW_Delta	106	13	erasure regressions
LF_HLW_Regression	69	7	all-rule regressions
LF_HLW_Report	304	4	diagnostic reporting

B Claim to theorem

Paper item	Compact	Exact
Theorem 4, forward	<code>theorem_4_forward</code>	—
Theorem 4, converse	<code>theorem_4_backward_fails</code>	<code>hl_theorem_4_backward_fails</code>
Proposition 5	<code>proposition_5</code>	—
Corollary 6	<code>corollary_6</code>	—
Proposition 7	<code>proposition_7</code>	<code>hl_proposition_7</code>
Theorem 10	<code>theorem_10</code>	<code>hl_theorem_10</code>
Proposition 18	<code>proposition_18</code>	—
Theorem 22	<code>theorem_22</code>	<code>hl_thm22_*</code>
Lemma 23	<code>renaming</code>	<code>hlDerivationOK_rename</code>
Corollary 24	<code>renaming_open</code>	<code>hlRenameDerivation_open_unchanged</code>
Conjecture 26 (corrected)	<code>conjecture_26</code>	<code>hlPaperCorrect_toFitch</code>
Conjecture 27	<code>conjecture_27_false</code>	<code>hl_conjecture_27_false</code>
Conjecture 28	<code>conjecture_28_false</code>	<code>hl_conjecture_28_false</code>
L1	<code>L1</code>	<code>hlToDerivation_total</code>
L2, L3	<code>L2_L3</code>	<code>hlClassifiedDerivationToFitch_nestedWellFormed</code>
L4	<code>L4_derivation</code>	<code>hlEmitDerivationFuel_ruleOK, ..._scopeOK</code>
Soundness	<code>lemmon_semantic_soundness</code>	<code>hlRuleOK_sound</code>